DATA STRUCTURE LABORATORY

C Language – Practical Programs

Short, separately organized programs with readable indentation

Student Name	_____
Class / Division	_____
Roll Number	_____
College	_____

Notes

- Each program is independent. Save each code in a separate .c file and compile it separately.
- Use a C compiler such as GCC. Example: **gcc program.c -o program**
- Enter array sizes and values as requested. For Practical 1(b), roll numbers must be sorted.
- Practical 3 uses -1 as an empty hash-table marker; do not enter a telephone number of -1.
- Practical 9 treats flight routes as undirected. City names are represented by indexes 1 to n.

INDEX

Practical	Program / Topic
Practical 1(a)	Linear Search (unsorted roll numbers)
Practical 1(b)	Binary Search (sorted roll numbers)
Practical 2(a)	Selection Sort and top five scores
Practical 2(b)	Bubble Sort and top five scores
Practical 3(a)	Hash table with linear probing
Practical 3(b)	Hash table with quadratic probing
Practical 3(c)	Hash table with double hashing
Practical 4	Singly linked list: add, delete, count, display, concatenate
Practical 5	Cinema seat booking using doubly linked lists
Practical 6	Check balanced delimiters using a stack
Practical 7	Job queue using FIFO (enqueue and dequeue)
Practical 8(a)	Binary search tree insertion and tree details
Practical 8(b)	Binary search tree searching
Practical 8(c)	Binary search tree deletion
Practical 9	Flight routes graph using adjacency matrix and connectivity
Practical 10	Mini project: Student record system using hash table

Practical 1(a) – Linear Search (unsorted roll numbers)

Source code

```
#include <stdio.h>
int main(void) {
    int a[100], n, key, i, found = 0;
    printf("Number of students: ");
    scanf("%d", &n);
    printf("Enter roll numbers: ");
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Roll number to search: ");
    scanf("%d", &key);
    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            found = 1;
            break;
        }
    }
    if (found) printf("Student attended (position %d)\\n", i + 1);
    else printf("Student did not attend\\n");
    return 0;
}
```

Practical 1(b) – Binary Search (sorted roll numbers)

Source code

```
#include <stdio.h>
int main(void) {
    int a[100], n, key, low = 0, high, mid, found = 0, i;
    printf("Number of students: ");
    scanf("%d", &n);
    printf("Enter roll numbers in ascending order: ");
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Roll number to search: ");
    scanf("%d", &key);
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) { found = 1; break; }
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    if (found) printf("Student attended (position %d)\\n", mid + 1);
    else printf("Student did not attend\\n");
    return 0;
}
```

Practical 2(a) – Selection Sort and top five scores

Source code

```
#include <stdio.h>
int main(void) {
    float a[100], t;
    int n, i, j, min;
    printf("Number of students: ");
    scanf("%d", &n);
    printf("Enter percentages: ");
    for (i = 0; i < n; i++) scanf("%f", &a[i]);
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        t = a[i]; a[i] = a[min]; a[min] = t;
    }
    printf("Ascending order: ");
    for (i = 0; i < n; i++) printf("%.2f ", a[i]);
    printf("\\nTop scores: ");
    for (i = n - 1; i >= 0 && i >= n - 5; i--)
        printf("%.2f ", a[i]);
    return 0;
}
```

Practical 2(b) – Bubble Sort and top five scores

Source code

```
#include <stdio.h>
int main(void) {
    float a[100], t;
    int n, i, j;
    printf("Number of students: ");
    scanf("%d", &n);
    printf("Enter percentages: ");
    for (i = 0; i < n; i++) scanf("%f", &a[i]);
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
        }
    }
    printf("Ascending order: ");
    for (i = 0; i < n; i++) printf("%.2f ", a[i]);
    printf("\\nTop scores: ");
    for (i = n - 1; i >= 0 && i >= n - 5; i--)
        printf("%.2f ", a[i]);
    return 0;
}
```

Practical 3(a) – Hash table with linear probing

Source code

```
#include <stdio.h>
#define SIZE 11
int main(void) {
    long long h[SIZE], key;
    int i, j, n;
    for (i = 0; i < SIZE; i++) h[i] = -1;
    printf("Number of telephone numbers (max %d): ", SIZE);
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter number: "); scanf("%lld", &key);
        j = (int)(key % SIZE);
        while (h[j] != -1) j = (j + 1) % SIZE;
        h[j] = key;
    }
    printf("Index  Telephone\\n");
    for (i = 0; i < SIZE; i++)
        if (h[i] != -1) printf("%5d  %lld\\n", i, h[i]);
    printf("Search number: "); scanf("%lld", &key);
    j = (int)(key % SIZE);
    for (i = 0; i < SIZE && h[j] != -1; i++, j = (j + 1) % SIZE)
        if (h[j] == key) break;
    if (i < SIZE && h[j] == key) printf("Found at index %d\\n", j);
    else printf("Not found\\n");
    return 0;
}
```

Practical 3(b) – Hash table with quadratic probing

Source code

```
#include <stdio.h>
#define SIZE 11
int main(void) {
    long long h[SIZE], key;
    int i, j, n, k, placed;
    for (i = 0; i < SIZE; i++) h[i] = -1;
    printf("Number of telephone numbers (max %d): ", SIZE);
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter number: "); scanf("%lld", &key);
        j = (int)(key % SIZE); placed = 0;
        for (k = 0; k < SIZE; k++) {
            int p = (j + k * k) % SIZE;
            if (h[p] == -1) { h[p] = key; placed = 1; break; }
        }
        if (!placed) printf("Table full; number skipped\\n");
    }
    for (i = 0; i < SIZE; i++)
        if (h[i] != -1) printf("%d : %lld\\n", i, h[i]);
    return 0;
}
```

Practical 3(c) – Hash table with double hashing

Source code

```
#include <stdio.h>
#define SIZE 11
#define PRIME 7
int main(void) {
    long long h[SIZE], key;
    int i, j, n, step, k, placed;
    for (i = 0; i < SIZE; i++) h[i] = -1;
    printf("Number of telephone numbers (max %d): ", SIZE);
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter number: "); scanf("%lld", &key);
        j = (int)(key % SIZE);
        step = PRIME - (int)(key % PRIME); placed = 0;
        for (k = 0; k < SIZE; k++) {
            int p = (j + k * step) % SIZE;
            if (h[p] == -1) { h[p] = key; placed = 1; break; }
        }
        if (!placed) printf("Table full; number skipped\\n");
    }
    for (i = 0; i < SIZE; i++)
        if (h[i] != -1) printf("%d : %lld\\n", i, h[i]);
    return 0;
}
```

Practical 4 – Singly linked list: add, delete, count, display, concatenate

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node {
    int prn;
    char name[30];
    struct Node *next;
} Node;

Node *addEnd(Node *head, int p, char name[]) {
    Node *n = malloc(sizeof(Node)), *t = head;
    n->prn = p; strcpy(n->name, name); n->next = NULL;
    if (!head) return n;
    while (t->next) t = t->next;
    t->next = n;
    return head;
}
Node *deletePRN(Node *head, int p) {
    Node *t = head, *prev = NULL;
    while (t && t->prn != p) { prev = t; t = t->next; }
    if (!t) { printf("Member not found\\n"); return head; }
    if (!prev) head = t->next; else prev->next = t->next;
    free(t); return head;
}
void display(Node *h) {
    while (h) {
        printf("%d %s\\n", h->prn, h->name);
        h = h->next;
    }
}
int count(Node *h) {
    int c = 0; while (h) { c++; h = h->next; } return c;
}
Node *concat(Node *a, Node *b) {
    Node *t;
    if (!a) return b;
    for (t = a; t->next; t = t->next) {}
    t->next = b; return a;
}
int main(void) {
    Node *club = NULL, *division = NULL;
    int ch, p; char name[30];
    do {
        printf("\\n1 Add club member 2 Delete member 3 Display/count");
        printf("\\n4 Add division member 5 Concatenate 0 Exit\\nChoice: ");
        scanf("%d", &ch);
        if (ch == 1 || ch == 4) {
            printf("PRN and name: "); scanf("%d %29s", &p, name);
            if (ch == 1) club = addEnd(club, p, name);
            else division = addEnd(division, p, name);
        } else if (ch == 2) {
            printf("PRN to delete: "); scanf("%d", &p);
            club = deletePRN(club, p);
        } else if (ch == 3) {
            display(club); printf("Total members: %d\\n", count(club));
        } else if (ch == 5) {
            club = concat(club, division); division = NULL;
            printf("Lists concatenated\\n");
        }
    } while (ch != 0);
    return 0;
}
```

Practical 5 – Cinema seat booking using doubly linked lists

Source code

```
#include <stdio.h>
#include <stdlib.h>
#define ROWS 10
#define SEATS 7
typedef struct Seat {
    int no, booked;
    struct Seat *prev, *next;
} Seat;
Seat *row[ROWS];

void initialize(void) {
    int r, s;
    for (r = 0; r < ROWS; r++) {
        Seat *last = NULL;
        for (s = 1; s <= SEATS; s++) {
            Seat *p = calloc(1, sizeof(Seat));
            p->no = s; p->prev = last;
            if (last) last->next = p;
            else row[r] = p;
            last = p;
        }
    }
}
void display(void) {
    int r; Seat *p;
    for (r = 0; r < ROWS; r++) {
        printf("Row %d: ", r + 1);
        for (p = row[r]; p; p = p->next)
            printf("%d:%s ", p->no, p->booked ? "Booked" : "Free");
        printf("\\n");
    }
}
void book(int r, int s) {
    Seat *p;
    if (r < 1 || r > ROWS || s < 1 || s > SEATS) return;
    for (p = row[r - 1]; p && p->no != s; p = p->next) {}
    if (!p) return;
    if (p->booked) printf("Already booked\\n");
    else { p->booked = 1; printf("Seat booked\\n"); }
}
void cancel(int r, int s) {
    Seat *p;
    if (r < 1 || r > ROWS || s < 1 || s > SEATS) return;
    for (p = row[r - 1]; p && p->no != s; p = p->next) {}
    if (p && p->booked) { p->booked = 0; printf("Booking cancelled\\n"); }
    else printf("Seat is not booked\\n");
}
int main(void) {
    int ch, r, s; initialize();
    do {
        printf("\\n1 Show seats 2 Book 3 Cancel 0 Exit\\nChoice: ");
        scanf("%d", &ch);
        if (ch == 2 || ch == 3) {
            printf("Row (1-10) and seat (1-7): ");
            scanf("%d%d", &r, &s);
            if (ch == 2) book(r, s); else cancel(r, s);
        } else if (ch == 1) display();
    } while (ch != 0);
    return 0;
}
```

Practical 6 – Check balanced delimiters using a stack

Source code

```
#include <stdio.h>
#define MAX 200
int main(void) {
    char exp[MAX], st[MAX];
    int top = -1, i, ok = 1;
    printf("Enter expression: ");
    fgets(exp, sizeof(exp), stdin);
    for (i = 0; exp[i] != '\\0'; i++) {
        char c = exp[i];
        if (c == '(' || c == '[' || c == '{') st[++top] = c;
        else if (c == ')' || c == ']' || c == '}') {
            if (top == -1) { ok = 0; break; }
            if ((c == ')' && st[top] != '(') ||
                (c == ']' && st[top] != '[') ||
                (c == '}' && st[top] != '{')) { ok = 0; break; }
            top--;
        }
    }
    if (top != -1) ok = 0;
    if (ok) printf("Expression is balanced\\n");
    else printf("Expression is not balanced\\n");
    return 0;
}
```

Practical 7 – Job queue using FIFO (enqueue and dequeue)

Source code

```
#include <stdio.h>
#define MAX 50
int q[MAX], front = 0, rear = -1, count = 0;
void addJob(int job) {
    if (count == MAX) { printf("Queue full\\n"); return; }
    rear = (rear + 1) % MAX; q[rear] = job; count++;
    printf("Job added\\n");
}
void deleteJob(void) {
    if (count == 0) { printf("Queue empty\\n"); return; }
    printf("Processed job: %d\\n", q[front]);
    front = (front + 1) % MAX; count--;
}
void display(void) {
    int i;
    if (!count) { printf("Queue empty\\n"); return; }
    for (i = 0; i < count; i++)
        printf("%d ", q[(front + i) % MAX]);
    printf("\\n");
}
int main(void) {
    int ch, job;
    do {
        printf("\\n1 Add job 2 Delete job 3 Display 0 Exit\\nChoice: ");
        scanf("%d", &ch);
        if (ch == 1) { printf("Job ID: "); scanf("%d", &job); addJob(job); }
        else if (ch == 2) deleteJob();
        else if (ch == 3) display();
    } while (ch != 0);
    return 0;
}
```

Practical 8(a) – Binary search tree insertion and tree details

Source code

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
Node *insert(Node *root, int x) {
    if (!root) {
        root = malloc(sizeof(Node));
        root->data = x; root->left = root->right = NULL;
    } else if (x < root->data) root->left = insert(root->left, x);
    else if (x > root->data) root->right = insert(root->right, x);
    return root;
}
int height(Node *r) {
    int a, b;
    if (!r) return 0;
    a = height(r->left); b = height(r->right);
    return 1 + (a > b ? a : b);
}
int minValue(Node *r) {
    while (r->left) r = r->left;
    return r->data;
}
void inorder(Node *r) {
    if (r) { inorder(r->left); printf("%d ", r->data); inorder(r->right); }
}
void mirror(Node *r) {
    Node *t;
    if (!r) return;
    t = r->left; r->left = r->right; r->right = t;
    mirror(r->left); mirror(r->right);
}
int main(void) {
    Node *root = NULL;
    int n, x, i;
    printf("Number of nodes: "); scanf("%d", &n);
    printf("Enter values: ");
    for (i = 0; i < n; i++) { scanf("%d", &x); root = insert(root, x); }
    printf("Inorder: "); inorder(root);
    if (root) printf("\\nMinimum value: %d", minValue(root));
    printf("\\nLongest root-to-leaf path (nodes): %d\\n", height(root));
    mirror(root);
    printf("After swapping left/right at every node: "); inorder(root);
    return 0;
}
```

Practical 8(b) – Binary search tree searching

Source code

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
Node *insert(Node *r, int x) {
    if (!r) {
        r = malloc(sizeof(Node)); r->data = x;
        r->left = r->right = NULL;
    } else if (x < r->data) r->left = insert(r->left, x);
    else if (x > r->data) r->right = insert(r->right, x);
    return r;
}
int search(Node *r, int key) {
    while (r) {
        if (key == r->data) return 1;
        r = key < r->data ? r->left : r->right;
    }
    return 0;
}
int main(void) {
    Node *root = NULL;
    int n, x, key, i;
    printf("Number of nodes: "); scanf("%d", &n);
    printf("Enter values: ");
    for (i = 0; i < n; i++) { scanf("%d", &x); root = insert(root, x); }
    printf("Value to search: "); scanf("%d", &key);
    if (search(root, key)) printf("Value found\\n");
    else printf("Value not found\\n");
    return 0;
}
```

Practical 8(c) – Binary search tree deletion

Source code

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
Node *insert(Node *r, int x) {
    if (!r) {
        r = malloc(sizeof(Node)); r->data = x;
        r->left = r->right = NULL;
    } else if (x < r->data) r->left = insert(r->left, x);
    else if (x > r->data) r->right = insert(r->right, x);
    return r;
}
Node *minNode(Node *r) {
    while (r && r->left) r = r->left;
    return r;
}
Node *deleteNode(Node *r, int x) {
    Node *t;
    if (!r) return NULL;
    if (x < r->data) r->left = deleteNode(r->left, x);
    else if (x > r->data) r->right = deleteNode(r->right, x);
    else {
        if (!r->left) { t = r->right; free(r); return t; }
        if (!r->right) { t = r->left; free(r); return t; }
        t = minNode(r->right); r->data = t->data;
        r->right = deleteNode(r->right, t->data);
    }
    return r;
}
void inorder(Node *r) {
    if (r) { inorder(r->left); printf("%d ", r->data); inorder(r->right); }
}
int main(void) {
    Node *root = NULL;
    int n, x, key, i;
    printf("Number of nodes: "); scanf("%d", &n);
    printf("Enter values: ");
    for (i = 0; i < n; i++) { scanf("%d", &x); root = insert(root, x); }
    printf("Value to delete: "); scanf("%d", &key);
    root = deleteNode(root, key);
    printf("Inorder after deletion: "); inorder(root);
    return 0;
}
```

Practical 9 – Flight routes graph using adjacency matrix and connectivity

Source code

```
#include <stdio.h>
#define MAX 20
int main(void) {
    int a[MAX][MAX] = {0}, seen[MAX] = {0};
    int n, e, i, u, v, start, count = 0, q[MAX], f = 0, r = 0, x;
    printf("Number of cities: "); scanf("%d", &n);
    printf("Number of flight routes: "); scanf("%d", &e);
    printf("Enter each route as city-index pair (1 to n):\\n");
    for (i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        if (u >= 1 && u <= n && v >= 1 && v <= n)
            a[u - 1][v - 1] = a[v - 1][u - 1] = 1;
    }
    printf("Adjacency matrix:\\n");
    for (u = 0; u < n; u++) {
        for (v = 0; v < n; v++) printf("%d ", a[u][v]);
        printf("\\n");
    }
    printf("Start city index: "); scanf("%d", &start);
    if (start < 1 || start > n) return 0;
    q[r++] = start - 1; seen[start - 1] = 1;
    while (f < r) {
        x = q[f++]; count++;
        for (i = 0; i < n; i++)
            if (a[x][i] && !seen[i]) {
                seen[i] = 1; q[r++] = i;
            }
    }
    if (count == n) printf("Graph is connected\\n");
    else printf("Graph is not connected\\n");
    return 0;
}
```

Practical 10 – Mini project: Student record system using hash table

Source code

```
#include <stdio.h>
#define SIZE 101
typedef struct {
    int roll;
    float percentage;
} Student;
Student table[SIZE];

int hash(int roll) { return roll % SIZE; }
void add(int roll, float per) {
    int i, p = hash(roll);
    for (i = 0; i < SIZE; i++, p = (p + 1) % SIZE) {
        if (table[p].roll == -1 || table[p].roll == roll) {
            table[p].roll = roll; table[p].percentage = per;
            printf("Record saved\\n"); return;
        }
    }
    printf("Hash table full\\n");
}
void find(int roll) {
    int i, p = hash(roll);
    for (i = 0; i < SIZE; i++, p = (p + 1) % SIZE) {
        if (table[p].roll == -1) break;
        if (table[p].roll == roll) {
            printf("Roll: %d  Percentage: %.2f\\n",
                   table[p].roll, table[p].percentage);
            return;
        }
    }
    printf("Record not found\\n");
}
int main(void) {
    int i, ch, roll; float per;
    for (i = 0; i < SIZE; i++) table[i].roll = -1;
    do {
        printf("\\n1 Add/update 2 Search 0 Exit\\nChoice: ");
        scanf("%d", &ch);
        if (ch == 1) {
            printf("Roll number and percentage: ");
            scanf("%d%f", &roll, &per); add(roll, per);
        } else if (ch == 2) {
            printf("Roll number: "); scanf("%d", &roll); find(roll);
        }
    } while (ch != 0);
    return 0;
}
```